T3000 API & Protocol

BACnet Commands Reference · built Nov 13, 2026

BACnet Commands Reference

BACnet commands are the low-level protocol operations used to communicate with field devices. T3000 implements the BACnet protocol to read and write data from controllers.

Key Concepts:

- Commands are identified by constants (e.g., READINPUT_T3000)
- Data is read/written in chunks (groups)
- Blocking operations ensure synchronous communication

Overview

T3000's C++ core implements BACnet MS/TP and BACnet/IP protocols for device communication. Each data type has specific read and write commands.

Read Commands

Input Commands

READINPUT_T3000

- **Purpose:** Read analog/digital inputs
- **Structure:** `Str_in_point` (sizeof = 46 bytes)
- **Max Count:** `BAC_INPUT_ITEM_COUNT` (64)
- **Group Size:** `BAC_READ_INPUT_GROUP_NUMBER` (8)
- **Usage:**

```cpp
GetPrivateDataSaveSPBlocking(
    objectinstance,
    READINPUT_T3000,
    start_index,
    end_index,
    sizeof(Str_in_point),
    4  // retry count
);
```

Output Commands

READOUTPUT_T3000

- **Purpose:** Read analog/digital outputs
- **Structure:** `Str_out_point` (sizeof = 46 bytes)
- **Max Count:** `BAC_OUTPUT_ITEM_COUNT` (64)
- **Group Size:** `BAC_READ_OUTPUT_GROUP_NUMBER` (8)

Variable Commands

READVARIABLE_T3000

- **Purpose:** Read user variables
- **Structure:** `Str_variable_point` (sizeof = 46 bytes)
- **Max Count:** `BAC_VARIABLE_ITEM_COUNT` (64)
- **Group Size:** `BAC_READ_VARIABLE_GROUP_NUMBER` (8)

Program Commands

READPROGRAM_T3000

- **Purpose:** Read control programs
- **Structure:** `Str_program_point` (sizeof = 37 bytes)
- **Max Count:** `BAC_PROGRAM_ITEM_COUNT` (16)
- **Group Size:** `BAC_READ_PROGRAM_GROUP_NUMBER` (10)

Controller Commands

READCONTROLLER_T3000

- **Purpose:** Read PID controllers
- **Structure:** `Str_controller_point` (sizeof = 24 bytes)

- **Max Count:** `BAC_PID_COUNT` (16)
- **Group Size:** `BAC_READ_PID_GROUP_NUMBER`

Monitor Commands

READMONITOR_T3000

- **Purpose:** Read trendlog monitors
- **Structure:** `Str_monitor_point` (sizeof = 133 bytes)
- **Max Count:** `BAC_MONITOR_COUNT` (12)
- **Group Size:** `BAC_READ_MONITOR_GROUP_NUMBER`

Schedule Commands

READWEEKLYSCHEDULE_T3000

- **Purpose:** Read weekly schedules
- **Structure:** `Str_weekly_routine_point`
- **Max Count:** 32

READANNUALSCHEDULE_T3000

- **Purpose:** Read annual holidays
- **Structure:** `Str_annual_routine_point`
- **Max Count:** 32

Write Commands

WRITEINPUT_T3000

Write input configuration (calibration, range, filter).

WRITEOUTPUT_T3000

Write output values and modes.

WRITEVARIABLE_T3000

Write variable values.

WRITEPROGRAM_T3000

Write program code and control.

WRITECONTROLLER_T3000

Write PID parameters.

WRITEMONITOR_T3000

Write trendlog configuration.

Reading Patterns

Chunked Reading

Data is read in groups to prevent network overload:

```C++
// Example: Read all programs (16 total) in chunks of 10
int totalCount = 16;
int groupSize = 10;  // BAC_READ_PROGRAM_GROUP_NUMBER
int groups = (totalCount + groupSize - 1) / groupSize;  // = 2 groups

for (int i = 0; i < groups; i++)
{
    int start = i * groupSize;  // Group 0: 0, Group 1: 10
    int end = min(start + groupSize - 1, totalCount - 1);  // Group 0: 9, Group 1: 15

    GetPrivateDataSaveSPBlocking(
        objectinstance,
        READPROGRAM_T3000,
        start,
        end,
        sizeof(Str_program_point),
        4
    );
}
```

Blocking vs Non-Blocking

GetPrivateDataSaveSPBlocking() - Synchronous (used in WebView messages)

- Waits for device response
- Retries on timeout (up to 4 times)
- Returns > 0 on success, < 0 on failure

GetPrivateData_Blocking() - Synchronous (used in refresh threads)

- Similar to above but different retry logic

Post_Refresh_Message() - Asynchronous (used in UI refresh)

- Queues read request
- Returns immediately
- Result handled in callback

Register Mapping

Each BACnet point type maps to specific Modbus registers:

Type	Start Register	Size (registers)
Inputs	`BAC_IN_START_REG` (11472)	23 per input
Outputs	`BAC_OUT_START_REG` (12945)	23 per output
Variables	`BAC_VAR_START_REG` (14417)	23 per variable
Programs	`BAC_PRG_START_REG` (15503)	19 per program
Controllers	`BAC_PID_START_REG` (15807)	12 per controller

Timing Parameters

```C++
#define SEND_COMMAND_DELAY_TIME 200  // Delay between commands (ms)
```

Recommended delays:

- Between sequential reads: 200ms
- After write commands: 200ms
- Retry timeout: 3000ms (3 seconds)

Error Codes

Return Value	Meaning
> 0	Success (bytes read)
-1	Timeout
-2	Invalid parameters
-3	Device not found
-4	Protocol error

Usage in Control Messages

Example: Action 17 (GET_WEBVIEW_LIST) for Programs

C++

```
case BAC_PRG:
{
    // Calculate groups
    int totalCount = entry_index_end - entry_index_start + 1;
    int groupSize = BAC_READ_PROGRAM_GROUP_NUMBER;
    int read_program_group = (totalCount + groupSize - 1) / groupSize;

    for (int i = 0; i < read_program_group; ++i)
    {
        int temp_start = entry_index_start + i * groupSize;
        int temp_end = min(temp_start + groupSize - 1, entry_index_end);

        // Execute BACnet read command
        if (GetPrivateDataSaveSPBlocking(
                entry_objectinstance,
                READPROGRAM_T3000,  // ← BACnet command
                (uint8_t)temp_start,
                (uint8_t)temp_end,
                sizeof(Str_program_point),
                4) > 0)
        {
            // Success - data now in s_Program_data buffer
            // Copy to global array g_Program_data
        }
    }
}
```

Command Constants Definition

Location: Defined in header files

The actual numeric values are defined in the T3000 codebase. Commands are used by name (constant) rather than hardcoded numbers.

See Also

- Data Structures - Point structure definitions

- Message 17: GET_WEBVIEW_LIST - Using read commands
- Platform Overview - Architecture